Code walkthrough

mux.v

Multiplexer

What it does	Chooses one of two 4-bit inputs
Where it works	used in IF and EX
Size	23 lines, explained in 2 small parts
Modules in this file	mux2
Calls (uses)	nothing (no other modules)
Called by	cpu_top.v - 3 times: instances PC_MUX, OPERAND_MUX, WB_MUX

A multiplexer (MUX) is a selector switch. It has two inputs and one output; a select line decides which input is passed through. The CPU uses this same small module three times: the PC MUX, the operand MUX and the write-back MUX.

Map of the file

Part	Lines	What this part is
1	1-15	Header comment
2	16-23	The whole module

> How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-15)

mux.v lines 1-15

```
 1  // ======================================================================
 2  // FILE: mux.v
 3  // 2-input, 4-bit multiplexer. A mux picks ONE of its inputs.
 4  //   sel = 0 -> y = d0
 5  //   sel = 1 -> y = d1
 6  //
 7  // The CPU uses this module three times:
 8  //   1. Operand MUX    : ALU input B = register Rs  or immediate
 9  //   2. Write-back MUX : register data = ALU result or memory data
10  //   3. PC MUX         : next PC = PC + 1  or jump address
11  //
12  // CONNECTIONS
13  //   Calls       : nothing (no other modules)
14  //   Called by   : cpu_top.v - 3 times: instances PC_MUX, OPERAND_MUX, WB_MUX
15  // ======================================================================
```

- The header explains what a MUX does and lists the three places it is used in the CPU. Comments start with `//` and are ignored by the tools.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · The whole module (lines 16-23)

mux.v lines 16-23

```
16  module mux2 (
17      input  [3:0] d0,
18      input  [3:0] d1,
19      input        sel,
20      output [3:0] y
21  );
22      assign y = (sel == 1'b1) ? d1 : d0;
23  endmodule
```

- The ports: two 4-bit data inputs `d0` and `d1`, a 1-bit select `sel`, and a 4-bit output `y`.
- `assign y = (sel == 1'b1) ? d1 : d0;` reads as: "if sel is 1, y is d1, otherwise y is d0". The `? :` operator is a one-line if/else.
- `assign` makes it combinational: y changes immediately when an input changes; there is no clock.

> **In short:** Three copies of this 6-line module steer all the data in the CPU. **Example** (PC MUX): d0 = 0011 (PC + 1), d1 = 1001 (jump address). If sel = 0 then y = 0011 (next instruction); if sel = 1 then y = 1001 (jump).